

CS325 Compilers Coursework Report

MiniC Compiler Implementation

Student ID: u5502438

November 24, 2025

1 Grammar Transformation and LL(k) Analysis

The original MiniC grammar contained left recursion in expression productions, preventing LL(k) parsing. Using the standard transformation ($A \rightarrow A\alpha \mid \beta$ becomes $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \epsilon$), I restructured expressions into a seven-level precedence hierarchy ($or_expr \rightarrow and_expr \rightarrow \dots \rightarrow unary_expr$). Rather than explicit prime non-terminals, I implemented iterative parsing with while-loops, maintaining left-to-right associativity whilst avoiding recursion overhead.

The transformed grammar (Appendix) is **LL(2)** because the *expr* production requires two-token lookahead: `IDENT` could indicate assignment (`x = expr`), array access (`x[i]`), function call (`x(args)`), or variable reference. A deque-based token buffer with `peekNextToken()` enables predictive parsing without backtracking. All other productions are LL(1). Complete FIRST and FOLLOW sets in the Appendices confirm no FIRST/FIRST or FIRST/FOLLOW conflicts; ϵ -productions verified against FOLLOW sets.

2 Parser Design Decisions

Architecture: Predictive recursive descent chosen over table-driven LL for: (1) direct grammar-to-code mapping enhancing maintainability, (2) simpler error message generation, (3) natural AST construction, (4) industry-standard approach. Each non-terminal maps to one parsing function. Token management uses `std::deque<TOKEN>` enabling arbitrary lookahead via `peekToken(offset)`, essential for LL(2) parsing.

Error handling: Fail-fast approach terminates on first error with detailed diagnostics. In batch compilation, comprehensive single error messages outweigh partial parse recovery. Enhanced error reporting includes: (1) source line display with caret pointer indicating error location, (2) “Did you mean?” suggestions using Levenshtein distance (edit distance ≤ 2) for undefined variables, (3) contextual help text, (4) colour-coded output. Errors accumulate in `ErrorLog` for end-of-compilation display.

AST design: All nodes inherit from `ASTnode` with virtual `codegen()` and `to_string()` methods. Smart pointers (`std::unique_ptr`) ensure automatic memory management. Post-order traversal produces correct SSA form: children generate code before parents.

3 Semantic Analysis Implementation

Type system: Three-type hierarchy `bool` $\rightarrow$ `int` $\rightarrow$ `float` mapped to LLVM types `i1` $\rightarrow$ `i32` $\rightarrow$ `float`. Widening conversions (information-preserving) allowed implicitly: `bool` $\rightarrow$ `int` via `CreateZExt`, `int` $\rightarrow$ `float` via `CreateSIToFP`. Narrowing conversions prohibited in assignments but allowed in boolean contexts (conditionals, logical operators), reflecting C semantics. Binary operations automatically promote operands via `promoteTypes()` to common type. Critical fix: normalized LLVM double literals to float using `CreateFPTrunc` before promotion.

Symbol tables: Three-level structure reflecting LLVM architecture: (1) `NamedValues` maps local variables to `AllocaInst*`, (2) `GlobalValues` maps globals to `GlobalVariable*`, (3)

`SymbolTable` stores type information for error reporting. Resolution checks local scope first, then global. Shadowing permitted. Block entry saves bindings in `OldBindings`; block exit restores previous scope, implementing lexical scoping with automatic cleanup.

Semantic checks: Type checking during code generation rather than separate pass. Variable lookups via `checkVariableInScope()` validate existence before use. Function calls verify argument count and types match parameters. Array dimensions validated at declaration. Return type consistency enforced.

4 Code Generation Strategy

Memory model: All variables use explicit memory: `alloca` for locals (in entry block via `CreateEntryBlockAlloca()`), `GlobalVariable` for globals. Variable access via `load/store` instructions. Simpler than SSA/Phi construction for control flow; enables LLVM optimisation passes to handle promotion. Each `codegen()` returns `Value*` representing SSA register.

Control flow: If-then-else creates three basic blocks (then/else/merge) with `CreateCondBr`; both branches terminate with `CreateBr` to merge. While loops use header (condition evaluation), body, and after blocks; body branches back to header. Structured approach ensures proper block termination.

Arrays (Part 3): Multi-dimensional arrays (1D/2D/3D) use nested LLVM types: `int [10] [5]` becomes `[10 x [5 x i32]]`. Array access uses `GetElementPtr` with index list: first index 0 (pointer-to-array decay), subsequent indices navigate nested structure. For `arr[i][j]`, GEP receives `[0, i, j]`. Array parameters follow C semantics: decay to pointers (`int arr[10]` becomes `int*`), keeping trailing dimensions for multi-dimensional arrays.

Debug infrastructure: Four debug levels (`NONE`, `USER`, `PARSER`, `CODEGEN`, `VERBOSE`) via `-d` flag or `MCCOMP_DEBUG` environment variable. Parser tracing shows function entry/exit with call-depth indentation. Symbol table dumping displays variables, types, and current context for debugging semantic issues.

5 Sources Consulted

LLVM Tutorial Chapters 2-8: AST construction, code generation, control flow, mutable variables (<https://llvm.org/docs/tutorial/>); LLVM Language Reference Manual: instruction semantics, type system (<https://llvm.org/docs/LangRef.html>); LLVM Doxygen: IRBuilder API, Type classes (<https://llvm.org/doxygen/>); LLVM `GetElementPtr` FAQ: array indexing semantics; Aho, Sethi, Ullman, *Compilers* 2nd ed.: LL(k) parsing, left recursion elimination, FIRST/FOLLOW computation; ISO/IEC 9899:1999 (C99): type conversion rules, operator precedence.

6 Known Limitations

Error recovery: Parser terminates on first error; no synchronisation implemented. Production compilers use recovery heuristics to continue parsing. **Optimisation:** Generated IR unoptimised; no LLVM passes applied. **Type system:** Strict narrowing prohibition rejects some valid C implicit casts (e.g., `float` to `int` in initialisation). **Forward declarations:** Require `extern` keyword; implicit declarations unsupported. **Array bounds:** No runtime or compile-time bounds checking.

A Final Grammar

Grammar

$program \rightarrow extern_list\ decl_list \mid decl_list$
 $extern_list \rightarrow extern\ extern_list_prime$
 $extern_list_prime \rightarrow extern\ extern_list_prime \mid \varepsilon$
 $extern \rightarrow \mathbf{extern}\ type_spec\ IDENT\ (params) ;$
 $decl_list \rightarrow decl\ decl_list_prime$
 $decl_list_prime \rightarrow decl\ decl_list_prime \mid \varepsilon$
 $decl \rightarrow var_decl \mid fun_decl$
 $var_decl \rightarrow type_spec\ init_decl ;$
 $init_decl \rightarrow var \mid var = exp$
 $fun_decl \rightarrow type_spec\ IDENT\ (params)\ compound_st$
 $params \rightarrow param_list \mid \mathbf{void} \mid \varepsilon$
 $param_list \rightarrow param\ param_list_prime$
 $param_list_prime \rightarrow ,\ param\ param_list_prime \mid \varepsilon$
 $param \rightarrow type_spec\ var$
 $type_spec \rightarrow \mathbf{int} \mid \mathbf{float} \mid \mathbf{void}$
 $var \rightarrow IDENT\ var_prime$
 $var_prime \rightarrow [exp] \mid \varepsilon$
 $stmts \rightarrow stmt\ stmts \mid \varepsilon$
 $stmt \rightarrow exp_stmt \mid compound_st \mid selection_st \mid iteration_st \mid return_st$
 $exp_stmt \rightarrow exp ; \mid ;$
 $compound_st \rightarrow \{ local_decls\ stmts \}$
 $local_decls \rightarrow var_decl\ local_decls \mid \varepsilon$
 $selection_st \rightarrow \mathbf{if}\ (exp)\ stmt\ selection_tail$
 $selection_tail \rightarrow \mathbf{else}\ stmt \mid \varepsilon$
 $iteration_st \rightarrow \mathbf{while}\ (exp)\ stmt$
 $return_st \rightarrow \mathbf{return}\ return_tail ;$
 $return_tail \rightarrow exp \mid \varepsilon$
 $exp \rightarrow mutable\ exp_tail \mid simple_exp$
 $exp_tail \rightarrow =\ exp \mid simple_exp_prime$
 $mutable \rightarrow var$
 $simple_exp \rightarrow additive_exp\ rel_exp_prime$
 $rel_exp_prime \rightarrow relop\ additive_exp \mid \varepsilon$
 $relop \rightarrow \leq \mid < \mid > \mid \geq \mid == \mid !=$
 $additive_exp \rightarrow term\ add_exp_prime$
 $add_exp_prime \rightarrow addop\ term\ add_exp_prime \mid \varepsilon$
 $addop \rightarrow + \mid -$
 $term \rightarrow factor\ mul_exp_prime$

$mul_exp_prime \rightarrow mulop\ factor\ mul_exp_prime \mid \varepsilon$
 $mulop \rightarrow * \mid / \mid \%$
 $factor \rightarrow (exp) \mid var \mid call \mid NUM$
 $call \rightarrow IDENT\ (args)$
 $args \rightarrow arg_list \mid \varepsilon$
 $arg_list \rightarrow exp\ arg_list_prime$
 $arg_list_prime \rightarrow ,\ exp\ arg_list_prime \mid \varepsilon$

FIRST Sets

$FIRST(program) = \{extern, int, float, void\}$
 $FIRST(extern_list) = \{extern\}$
 $FIRST(extern_list_prime) = \{extern, \varepsilon\}$
 $FIRST(extern) = \{extern\}$
 $FIRST(decl_list) = \{int, float, void\}$
 $FIRST(decl_list_prime) = \{int, float, void, \varepsilon\}$
 $FIRST(decl) = \{int, float, void\}$
 $FIRST(var_decl) = \{int, float, void\}$
 $FIRST(init_decl) = \{IDENT\}$
 $FIRST(fun_decl) = \{int, float, void\}$
 $FIRST(params) = \{int, float, void, \varepsilon\}$
 $FIRST(param_list) = \{int, float, void\}$
 $FIRST(param_list_prime) = \{., \varepsilon\}$
 $FIRST(param) = \{int, float, void\}$
 $FIRST(type_spec) = \{int, float, void\}$
 $FIRST(var) = \{IDENT\}$
 $FIRST(var_prime) = \{[, \varepsilon\}$
 $FIRST(stmts) = \{IDENT, (, NUM, \{, if, while, return, ;, \varepsilon\}$
 $FIRST(stmt) = \{IDENT, (, NUM, \{, if, while, return, ;\}$
 $FIRST(exp_stmt) = \{IDENT, (, NUM, ;\}$
 $FIRST(compound_st) = \{\{\}$
 $FIRST(local_decls) = \{int, float, void, \varepsilon\}$
 $FIRST(selection_st) = \{if\}$
 $FIRST(selection_tail) = \{else, \varepsilon\}$
 $FIRST(iteration_st) = \{while\}$
 $FIRST(return_st) = \{return\}$
 $FIRST(return_tail) = \{IDENT, (, NUM, \varepsilon\}$
 $FIRST(exp) = \{IDENT, (, NUM\}$
 $FIRST(exp_tail) = \{=, <=, <, >, >=, ==, !=, +, -, *, /, \%, \varepsilon\}$
 $FIRST(mutable) = \{IDENT\}$
 $FIRST(simple_exp) = \{IDENT, (, NUM\}$
 $FIRST(rel_exp_prime) = \{<=, <, >, >=, ==, !=, \varepsilon\}$
 $FIRST(relop) = \{<=, <, >, >=, ==, !=\}$
 $FIRST(additive_exp) = \{IDENT, (, NUM\}$
 $FIRST(add_exp_prime) = \{+, -, \varepsilon\}$

$\text{FIRST}(\text{addop}) = \{+, -\}$
 $\text{FIRST}(\text{term}) = \{\text{IDENT}, (, \text{NUM}\}$
 $\text{FIRST}(\text{mul_exp_prime}) = \{*, /, \%, \varepsilon\}$
 $\text{FIRST}(\text{mulop}) = \{*, /, \%$
 $\text{FIRST}(\text{factor}) = \{(, \text{IDENT}, \text{NUM}\}$
 $\text{FIRST}(\text{call}) = \{\text{IDENT}\}$
 $\text{FIRST}(\text{args}) = \{\text{IDENT}, (, \text{NUM}, \varepsilon\}$
 $\text{FIRST}(\text{arg_list}) = \{\text{IDENT}, (, \text{NUM}\}$
 $\text{FIRST}(\text{arg_list_prime}) = \{., \varepsilon\}$

FOLLOW Sets

$\text{FOLLOW}(\text{program}) = \{\$ \}$
 $\text{FOLLOW}(\text{extern_list}) = \{\text{int}, \text{float}, \text{void}\}$
 $\text{FOLLOW}(\text{extern_list_prime}) = \{\text{int}, \text{float}, \text{void}\}$
 $\text{FOLLOW}(\text{extern}) = \{\text{extern}, \text{int}, \text{float}, \text{void}\}$
 $\text{FOLLOW}(\text{decl_list}) = \{\$ \}$
 $\text{FOLLOW}(\text{decl_list_prime}) = \{\$ \}$
 $\text{FOLLOW}(\text{decl}) = \{\text{int}, \text{float}, \text{void}, \$ \}$
 $\text{FOLLOW}(\text{var_decl}) = \{\text{int}, \text{float}, \text{void}, \text{IDENT}, \{, \text{if}, \text{while}, \text{return}, ;, \}\}$
 $\text{FOLLOW}(\text{init_decl}) = \{;\}$
 $\text{FOLLOW}(\text{fun_decl}) = \{\text{int}, \text{float}, \text{void}, \$ \}$
 $\text{FOLLOW}(\text{params}) = \{)\}$
 $\text{FOLLOW}(\text{param_list}) = \{)\}$
 $\text{FOLLOW}(\text{param_list_prime}) = \{)\}$
 $\text{FOLLOW}(\text{param}) = \{), , \}$
 $\text{FOLLOW}(\text{type_spec}) = \{\text{IDENT}\}$
 $\text{FOLLOW}(\text{var}) = \{=, <=, <, >, >=, ==, !=, +, -, *, /, \%, ;,),], , \}$
 $\text{FOLLOW}(\text{var_prime}) = \{=, <=, <, >, >=, ==, !=, +, -, *, /, \%, ;,),], , \}$
 $\text{FOLLOW}(\text{stmts}) = \{\}$
 $\text{FOLLOW}(\text{stmt}) = \{\text{IDENT}, (, \text{NUM}, \{, \text{if}, \text{while}, \text{return}, ;, \}\}$
 $\text{FOLLOW}(\text{exp_stmt}) = \{\text{IDENT}, (, \text{NUM}, \{, \text{if}, \text{while}, \text{return}, ;, \}\}$
 $\text{FOLLOW}(\text{compound_st}) = \{\text{IDENT}, (, \text{NUM}, \{, \text{if}, \text{while}, \text{return}, ;, \}, \text{else}\}$
 $\text{FOLLOW}(\text{local_decls}) = \{\text{IDENT}, (, \text{NUM}, \{, \text{if}, \text{while}, \text{return}, ;, \}\}$
 $\text{FOLLOW}(\text{selection_st}) = \{\text{IDENT}, (, \text{NUM}, \{, \text{if}, \text{while}, \text{return}, ;, \}, \text{else}\}$
 $\text{FOLLOW}(\text{selection_tail}) = \{\text{IDENT}, (, \text{NUM}, \{, \text{if}, \text{while}, \text{return}, ;, \}\}$
 $\text{FOLLOW}(\text{iteration_st}) = \{\text{IDENT}, (, \text{NUM}, \{, \text{if}, \text{while}, \text{return}, ;, \}\}$
 $\text{FOLLOW}(\text{return_st}) = \{\text{IDENT}, (, \text{NUM}, \{, \text{if}, \text{while}, \text{return}, ;, \}\}$
 $\text{FOLLOW}(\text{return_tail}) = \{;\}$
 $\text{FOLLOW}(\text{exp}) = \{;,),], , \}$
 $\text{FOLLOW}(\text{exp_tail}) = \{;,),], , \}$
 $\text{FOLLOW}(\text{mutable}) = \{=, <=, <, >, >=, ==, !=, +, -, *, /, \%, ;,),], , \}$
 $\text{FOLLOW}(\text{simple_exp}) = \{;,),], , \}$
 $\text{FOLLOW}(\text{rel_exp_prime}) = \{;,),], , \}$
 $\text{FOLLOW}(\text{relop}) = \{\text{IDENT}, (, \text{NUM}\}$

$\text{FOLLOW}(\text{additive_exp}) = \{<=, <, >, >=, ==, !=, ;,),], , \}$
 $\text{FOLLOW}(\text{add_exp_prime}) = \{<=, <, >, >=, ==, !=, ;,),], , \}$
 $\text{FOLLOW}(\text{addop}) = \{\text{IDENT}, (, \text{NUM}\}$
 $\text{FOLLOW}(\text{term}) = \{+, -, <=, <, >, >=, ==, !=, ;,),], , \}$
 $\text{FOLLOW}(\text{mul_exp_prime}) = \{+, -, <=, <, >, >=, ==, !=, ;,),], , \}$
 $\text{FOLLOW}(\text{mulop}) = \{\text{IDENT}, (, \text{NUM}\}$
 $\text{FOLLOW}(\text{factor}) = \{*, /, \%, +, -, <=, <, >, >=, ==, !=, ;,),], , \}$
 $\text{FOLLOW}(\text{call}) = \{*, /, \%, +, -, <=, <, >, >=, ==, !=, ;,),], , \}$
 $\text{FOLLOW}(\text{args}) = \{ \}$
 $\text{FOLLOW}(\text{arg_list}) = \{ \}$
 $\text{FOLLOW}(\text{arg_list_prime}) = \{ \}$